

W8 - BEFORE PRACTICE

BLA PROJECT - REFACTORING



Learning objectives

- ✓ Reflect on BlaBla app state management, given the UI specification
- ✓ Refactor the previous version with repositories, global state, provider, model views

How to start?

- ✓ Copy the **START CODE** into your current repository and push it

BLA-200 - Start Code Week 8

- ✓ Each task of this practice shall be related to commit(s) including the **tasks ID**.

BLA-201 - Create repositories

How to submit?

- ✓ Once finished, submit on MS Team
 - ✓ Your GitHub repository URL
 - ✓ This document if needed



This practice focusses more **on clean code structure** rather than Flutter technical skills.
We will evaluate how well **you follow the coding conventions**, how you name your class, variables etc.

No code generated by AI is allowed

The objective of this practice is to **remove all static services** and replace them with

- ✓ **Repositories** (abstract design pattern for testability)
- ✓ **Global states**
- ✓ **Model views** for screens and optionally for genetic widgets



The bellow approach below is just an example.

You are free to follow your architecture if validated by the lecturer.

/model

 location
 ride
 ride_preference

/repositories

 locations_repository
 rides_repository
 ride_preference_repository

/ui/states

 ride_preferences_state <<  ride_preference_repository

/ui/screens

 home
  home_view_model <<  ride_preferences_state
  home_content

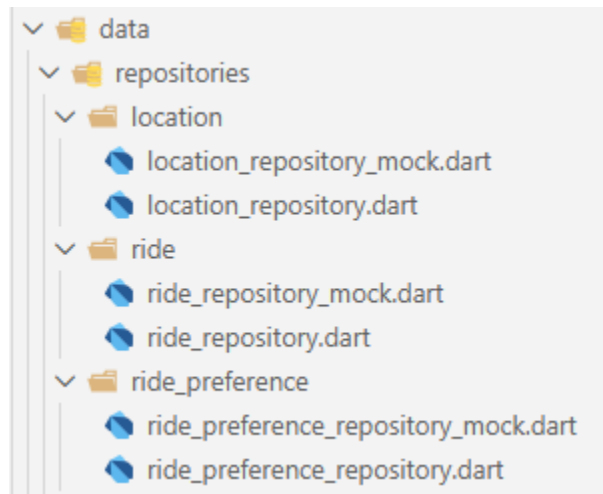
 ride_selection
  ride_selection_view_model <<  ride_preferences_state <<  rides_repository
  ride_selection_content

/ui/widgets/pickers

 bla_ride_preference_picker
 bla_location_picker <<  locations_repository
 bla_ride_preference_modal
 bla_seat_picker

BLA-201 – Repositories

- ✓ Create the bellow repositories
 - ✓ Create the abstract repositories first
 - ✓ Create mock repositories
 - ✓ You can refer to **fake data** instances for now (*locations, rides ...*)



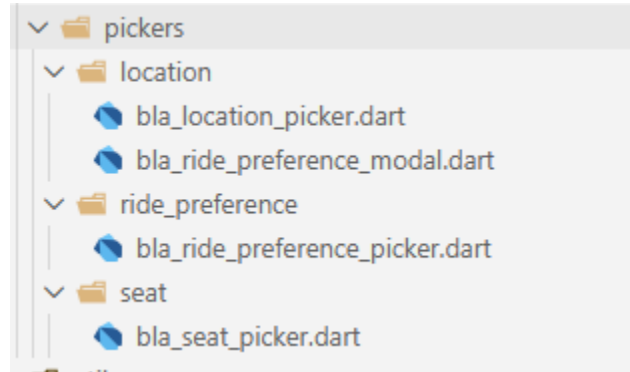
- ✓ Inject all repositories into the **root widget**
 - ✓ Use the same pattern (main_dev + main_common) to allow flexible environements.



Identify for each of the **previous static services**, which methods, attributes should:

- Be handled by a **repository**
- Be handled by a **global state** or **view model**

BLA-202 – Location Picker



- ✓ The location picker uses global `LocationRepository` to fetch the list of locations
- ✓ You can **remove the Location Service** (should not be used anymore)

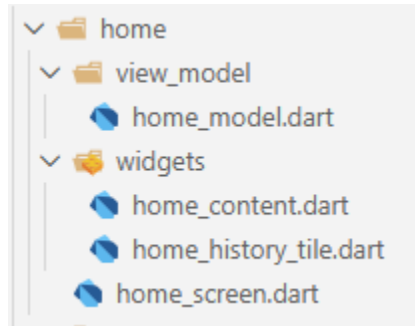
BLA-203 – Ride Preferences Global state

- ✓ Create the global state `RidePreferenceState` in the `ui/states`
- ✓ Inject this state into the **root widget**
- ✓ The `RidePreferenceState` shall use the `RidePreferencesRepository` to fetch the history
- ✓ The `RidePreferenceState` shall handle the selected preference

Select the preference

- ✓ Provide the method to select the current ride preference
 - ✓ The current preference shall be updated (if different from the previous one)
 - ✓ The history shall be updated with the new preference
 - ✓ The listener shall be notified

BLA-204 – Home Screen



Home Screen

- ✓ Follow the MVVM approach to separate the view content from the view model
- ✓ Pass the view model to the content

Home View Model

- ✓ The view model uses global state RidePreferenceState to get current preference and the history of past preferences.

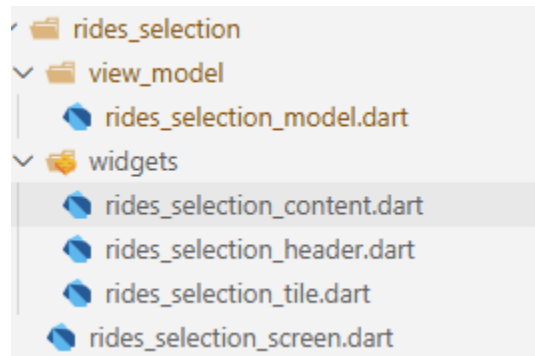
⚠ Don't forget to listen/unlisten to the global state to notify the screen upon any state change

Home Content

- ✓ You can build the HomeContent from the old screen code
- ✓ HomeContent gets its **view model** passed by constructor

⚠ HomeContent shall NOT be Stateful anymore → replace with notifications from the view model

BLA-205 – Ride Selection Screen



Ride Selection Screen

- ✓ Follow the MVVM approach to separate the view content from the view model
- ✓ Pass the view model to the content

Ride Selection View Model

- ✓ The model uses global RidePreferenceState to get current preference and the history of past preferences.
- ✓ The view model uses global RideRepository to fetch the list of rides

⚠ Don't forget to listen/unlisten to the global state to notify the screen upon any state change

❓ **Who compute the matching rides** (the rides matching with the current ride preference and the filter): the view? The model view? The repository?

Ride Selection Content

- ✓ You can build the RideSelectionContent from the old screen code
- ✓ RideSelectionContent gets its **view model** passed by constructor

⚠ RideSelectionContent shall NOT be Stateful anymore → replace with notifications from the view model